

Parser combinators



In this chapter, we'll work through the design of a combinator library for creating *parsers*. We'll use JSON parsing (<http://mng.bz/DpNA>) as a motivating use case. Like chapters 7 and 8, this chapter is not so much about parsing as it is about providing further insight into the process of functional design.

What is a parser?

A parser is a specialized program that takes unstructured data (such as text, or any kind of stream of symbols, numbers, or tokens) as input, and outputs a structured representation of that data. For example, we can write a parser to turn a comma-separated file into a list of lists, where the elements of the outer list represent the records, and the elements of each inner list represent the comma-separated fields of each record. Another example is a parser that takes an XML or JSON document and turns it into a tree-like data structure.

In a parser combinator library, like the one we'll build in this chapter, a parser doesn't have to be anything quite that complicated, and it doesn't have to parse entire documents. It can do something as elementary as recognizing a single character in the input. We then use combinators to assemble composite parsers from elementary ones, and still more complex parsers from those.

This chapter will introduce a design approach that we'll call *algebraic design*. This is just a natural evolution of what we've already been doing to different degrees in past chapters—designing our interface first, along with associated laws, and letting this guide our choice of data type representations.

At a few key points during this chapter, we'll give more open-ended exercises, intended to mimic the scenarios you might encounter when writing your own